



SISTEMAS DISTRIBUIDOS

Task1:Communication and middleware

2025-26

SALMA JADIANI
ALEJANDRO FERNÁNDEZ

ÍNDICE

1. Introducción

2. Diseño del Sistema

2.1. Modelos de tickets

2.2. Arquitecturas

2.3. Consistencia y Redis

3. Resultados y gráficas.

4. Análisis: Rendimiento, escalabilidad, impacto de la contención.

5. Conclusiones

1.Introducción

- **Objetivos:**

La gestión de venta de entradas para eventos de alta demanda (conciertos, eventos deportivos, etc.) representa uno de los desafíos más críticos en el diseño de sistemas distribuidos. El problema fundamental radica en la **conurrencia masiva**: miles de usuarios intentando adquirir un inventario limitado de asientos en el mismo segundo.

En esta práctica, se han implementado y analizado dos arquitecturas de comunicación para resolver este escenario:

1. **Comunicación Directa:** Basada en Pyro4 como middleware de RPC síncrono, con workers registrados en un Name Server y un balanceador de carga round-robin en el lado del cliente para distribuir las peticiones.
2. **Comunicación Indirecta:** Basada en un modelo asíncrono utilizando **RabbitMQ** como middleware de mensajería, donde el procesamiento se desacopla de la recepción mediante el uso de *Workers*.

El sistema utiliza **Redis** como motor de persistencia en memoria, aprovechando sus capacidades atómicas para garantizar que el estado de los tickets sea consistente en todo momento.

El desarrollo de este proyecto se ha guiado por los siguientes objetivos principales:

- **Garantizar la Consistencia de Datos:** Asegurar que, bajo ninguna circunstancia, un mismo asiento numerado sea vendido a dos clientes distintos (evitar el *overselling*).
- **Implementar Robustez y Resiliencia:** Diseñar el sistema de comunicación indirecta de modo que sea capaz de sobrevivir a fallos críticos. Esto incluye la persistencia de mensajes en la cola y el manejo de confirmaciones manuales para evitar la pérdida de transacciones si un procesador (*Worker*) se desconecta.
- **Analizar la Escalabilidad:** Evaluar cómo el rendimiento del sistema (medido en operaciones por segundo o *throughput*) mejora de forma dinámica al añadir más nodos de procesamiento.
- **Estudiar el Impacto de la Contención (Hotspots):** Observar y documentar el comportamiento del sistema cuando una gran mayoría de las peticiones se concentran en un subconjunto muy pequeño de recursos (asientos específicos), provocando cuellos de

botella en la base de datos.

- **Comparativa Crítica:** Determinar las ventajas y desventajas de cada modelo de comunicación en función de la carga de trabajo y los requisitos de disponibilidad.

2. Diseño del Sistema

2.1 Modelos de tickets

- **No numerados.**

Este modelo representa eventos donde las entradas no tienen un asiento asignado (ej. pista en un concierto).

- **Lógica de Implementación:** Se utiliza un contador atómico en Redis mediante el comando **INCR**.

```
tickets_vendidos = self.redis.incr("contador_tickets")
```

- **Gestión de Concurrencia:** El sistema comprueba si el valor del contador es inferior al aforo total antes de confirmar la venta.

```
tickets_vendidos <= 20000:
```

- **Rendimiento:** Al no haber colisiones por un recurso específico, la contención es mínima, permitiendo que múltiples Workers incrementen el contador simultáneamente sin bloqueos.
 - Numerados.

Este modelo es propenso a generar **Hotspots**, ya que cada petición solicita un identificador de asiento

- **Lógica de Implementación:** Se utiliza una estructura de clave-valor donde la clave es el identificador del asiento. La reserva se realiza mediante la operación atómica SET NX (Set if Not Exists).

```
reservado = self.redis.set(clave_asiento, cliente_id, nx=True)
```

- **Garantía de Exclusividad:** El uso de NX asegura que solo la primera petición que llegue a Redis consiga escribir su request_id en esa clave. Cualquier intento posterior para el mismo asiento será rechazado por la base de datos, devolviendo un error de "asiento ya vendido".
- **Impacto en el Diseño:** En los benchmarks de alta carga, muchas peticiones compiten por el mismo asiento (Hotspot), lo que obliga a

los Workers a procesar y descartar rápidamente las peticiones duplicadas para mantener la consistencia.

2.2 Arquitecturas

- **Directa (PYRO4)**

En este modelo, un objeto en una máquina (servidor) expone sus métodos para que puedan ser ejecutados por otra máquina (cliente) de forma transparente, como si fueran llamadas locales.

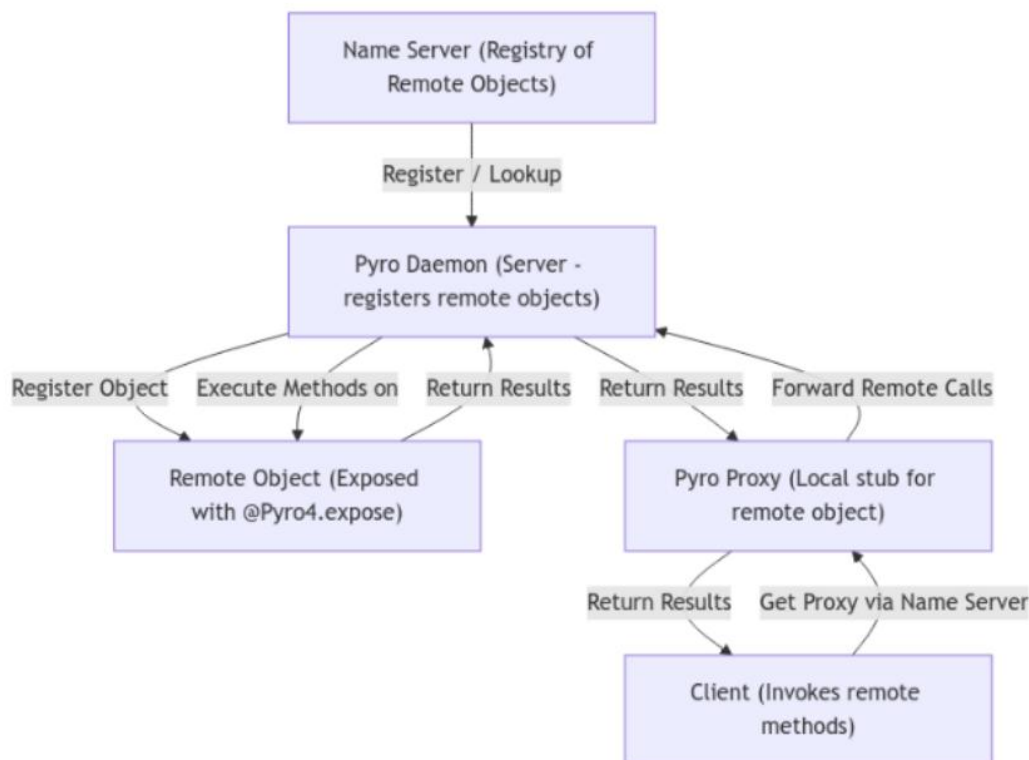


Fig. 1 - Arquitectura de Comunicación Directa (Pyro4)

Los componentes clave definidos en la teoría y aplicados en el proyecto son:

1. **Daemon:** Es el servidor encargado de escuchar las peticiones de red y despacharlas al objeto real.
2. **Proxy:** Es el "stub" o representante local en el cliente que encapsula la comunicación de red.
3. **Name Server:** Un registro centralizado donde los servidores publican sus nombres y los clientes buscan las direcciones (URIs).

En nuestro sistema, la comunicación directa se ha estructurado en tres niveles:

- Name Server: Utilizamos el Name Server de Pyro para desacoplar la ubicación física de los workers. En `server_pyro.py`, cada instancia se registra con un id. único `tickets.worker.1`
- El Servidor Directo -Worker: `server_pyro.py` implementa la clase `TicketWorker`, la cual está marcada con el decorador `@Pyro4.expose`. Esta clase actúa como un puente: recibe la llamada RPC del cliente y utiliza la lógica definida en `base.modelo_compra` para interactuar con el backend de Redis.
- El Cliente: El archivo `client_pyro.py` implementa un Balanceador de Carga en el Lado del Cliente (Client-side Load Balancing).
 - El cliente consulta al Name Server todos los objetos con el prefijo `tickets.worker.*`.
 - Reparte las peticiones del benchmark utilizando un algoritmo de Round Robin, enviando cada petición de compra a un worker diferente de la lista.

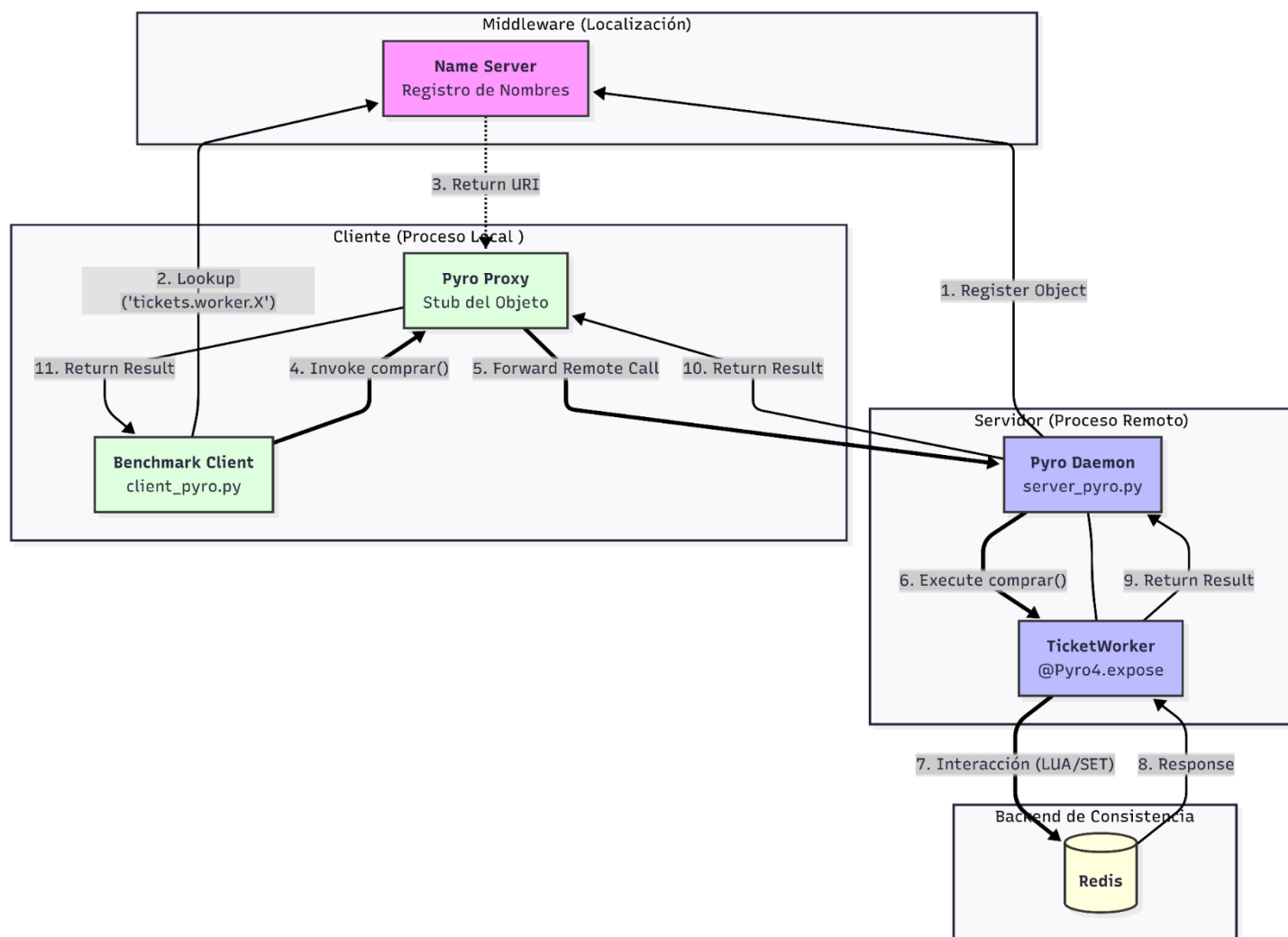


Fig. 2 - Balanceo de carga round-robin del lado del cliente

- Indirecta (RabbitMQ)

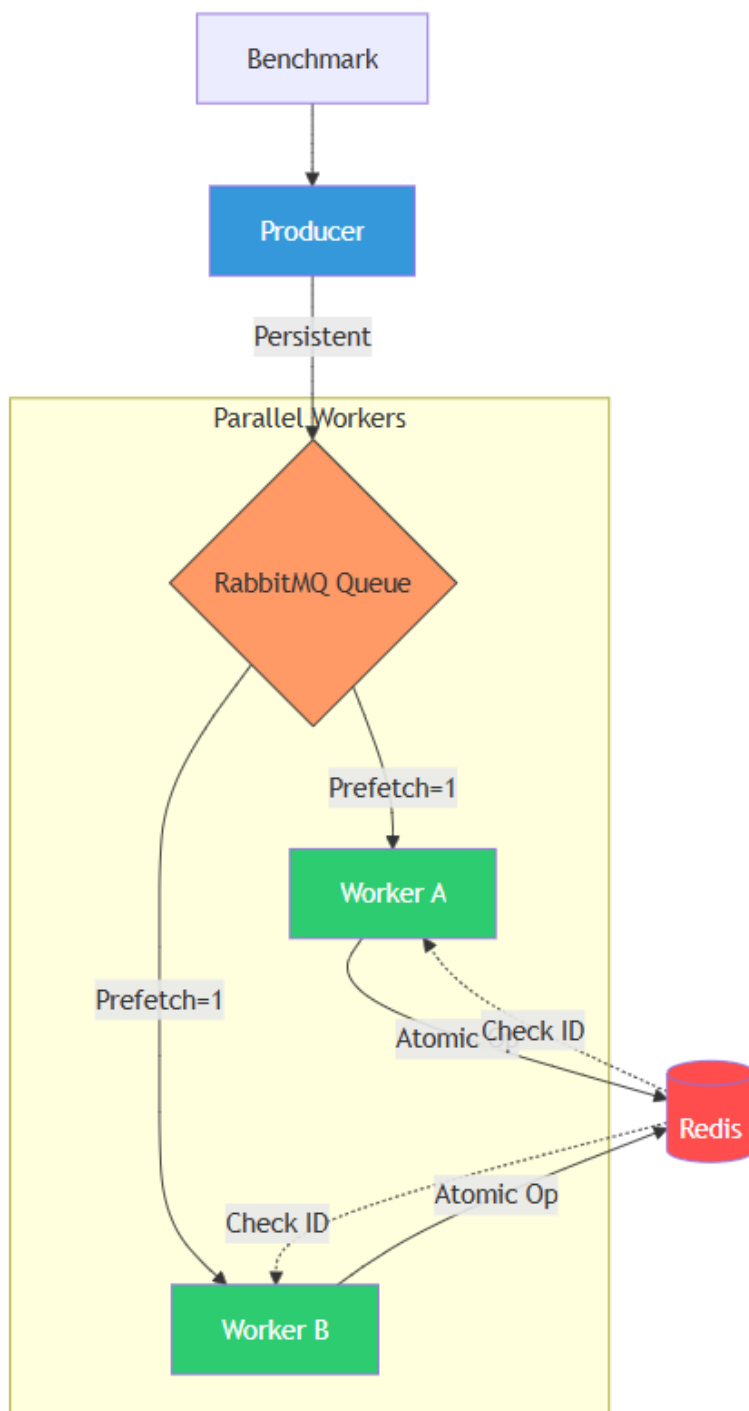


Fig. 3 - Arquitectura de Comunicación Indirecta (RabbitMQ)

En el modelo indirecto un **Producer** desacopla la carga del archivo de *benchmark* del procesamiento real y envía peticiones persistentes a una cola de RabbitMQ “cola_tickets”. El sistema escala horizontalmente mediante múltiples **Workers** que, bajo una política de Fair Dispatch (**Prefetch=1**) y confirmaciones manuales (**ACK**), consumen mensajes de

forma equilibrada, asegurando que ninguna tarea se pierda si un nodo falla. Finalmente, todos los Workers convergen en una base de datos Redis, donde se ejecutan operaciones atómicas (SET NX / INCR) y validaciones de idempotencia mediante el `request_id`, garantizando así la consistencia absoluta del inventario y evitando la sobreventa de tickets incluso en escenarios de alta contención o reintentos de red.

2.3 Consistencia, Redis.

Para garantizar la integridad del inventario, el sistema delega la exclusividad de las ventas a Redis, actuando frente a los múltiples Workers.

En el caso de asientos numerados, el código utiliza la instrucción `set(clave, valor, nx=True)`, donde el parámetro NX asegura que la reserva solo sea exitosa si el asiento no ha sido ocupado previamente por otro proceso.

Para los tickets no numerados, se emplea la función `incr(contador)`, que permite gestionar el aforo de forma segura mediante incrementos atómicos que evitan condiciones de carrera.

Finalmente, la idempotencia se asegura mediante un registro de `request_id` en Redis: antes de confirmar cualquier operación, el Worker verifica si dicho identificador ya existe, evitando que reintentos de red de RabbitMQ provoquen ventas duplicadas y garantizando que el estado final de la base de datos sea siempre consistente.

3. Resultados y gráficas

Para evaluar el rendimiento de ambas arquitecturas se usaron 4 benchmarks de distintos escenarios de reserva de asiento.

Para cada benchmark se midió: tiempo total, throughput (ops/seg) y éxito/fallo de las operaciones.

Benchmark 1: Entradas no numeradas (20.000 peticiones)

20.000 peticiones de compra para un aforo de 20.000 asientos sin numerar así que todas las peticiones son exitosas.

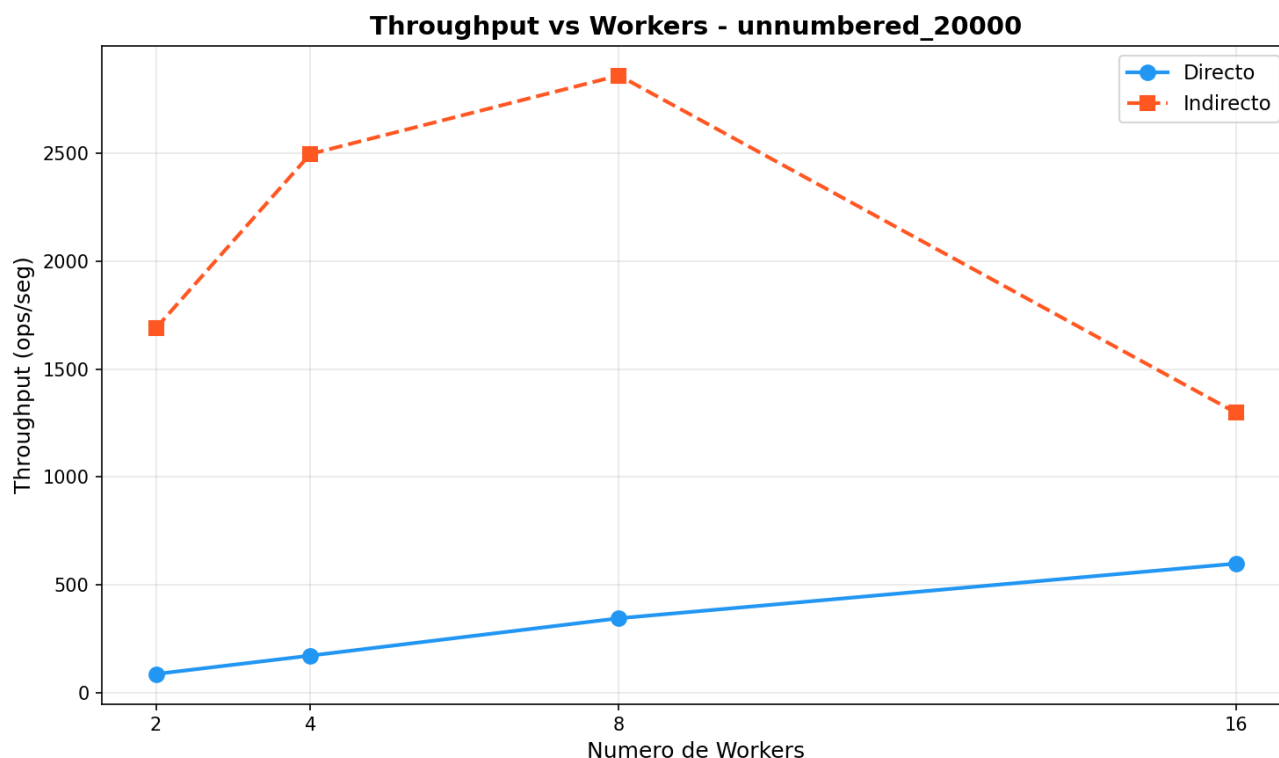


Fig. 4 - Throughput vs Workers (entradas no numeradas)

- **Modelo Directo (Pyro4):** Muestra una escalabilidad lineal pero modesta, creciendo uniformemente desde las 100 ops/seg (2 *workers*) hasta su máximo de 600 ops/seg (16 *workers*). Al tratarse de un modelo síncrono (RPC), el cliente debe bloquearse y esperar la respuesta de cada llamada. Aunque el balanceo *Round Robin* aprovecha los nuevos nodos, el rendimiento global se ve severamente limitado por el *overhead* de red y sobretodo los tiempos de espera síncronos.
- **Modelo Indirecto (RabbitMQ):** Este modelo multiplica el rendimiento, alcanzando un pico óptimo de casi 2.850 ops/seg con 8 *workers*. No obstante, al escalar a 16 *workers* el rendimiento cae a la mitad (~1.300 ops/seg). El cuello de botella puede resultar de la gestión masiva de confirmaciones manuales (ACK) y la política

Fair Dispatch (Prefetch=1) que saturan a RabbitMQ, penalizando el rendimiento para mantener la coordinación.

- **Conclusión:** El modelo indirecto es hasta 4.5 veces más rápido para cargas masivas ya que no requiere esperas síncronas, pero demuestra tener un techo de escalabilidad a partir de los 8 *workers*, donde la sobrecarga de gestión resulta contraproducente.

Benchmark 2: Entradas numeradas (25.997 peticiones, hotspot asiento 42)

25.997 peticiones para 20.000 asientos numerados, con el asiento 42 como hotspot (todas las primeras peticiones van al mismo asiento). Se venden 20.000 asientos únicos y 5.997 son rechazados por estar ya vendidos.



Fig. 5 - Throughput vs Workers (entradas numeradas, hotspot seat 42)

- **Modelo Directo (Pyro4):** Mantiene un comportamiento similar al escenario no numerado, escalando de manera lineal desde las 100 ops/seg hasta alcanzar unas 630 ops/seg con 16 workers (las peticiones rechazadas por el hotspot se procesan al mismo ritmo que las exitosas).

- **Modelo Indirecto (RabbitMQ):** Alcanza su rendimiento máximo de forma prematura con 4 workers (~1.800 ops/seg), pero sufre un bache al subir a 8 workers (~1.500 ops/seg) antes de recuperarse levemente con 16. Esto ocurre porque, al principio del benchmark.

Conclusión: El modelo indirecto es hasta 3 veces más rápido debido a la eficiencia asincronía, pero demuestra que un hotspot desplaza su punto óptimo a los 4 workers, sufriendo un bache por contención a los 8 workers antes de recuperarse por el paralelismo de los 16 workers.

Benchmark 3: Distribución uniforme (60.000 peticiones)

60.000 peticiones distribuidas uniformemente entre 20.000 asientos. Se esperan ~19.000 compras exitosas y ~41.000 rechazos por asiento ya vendido.

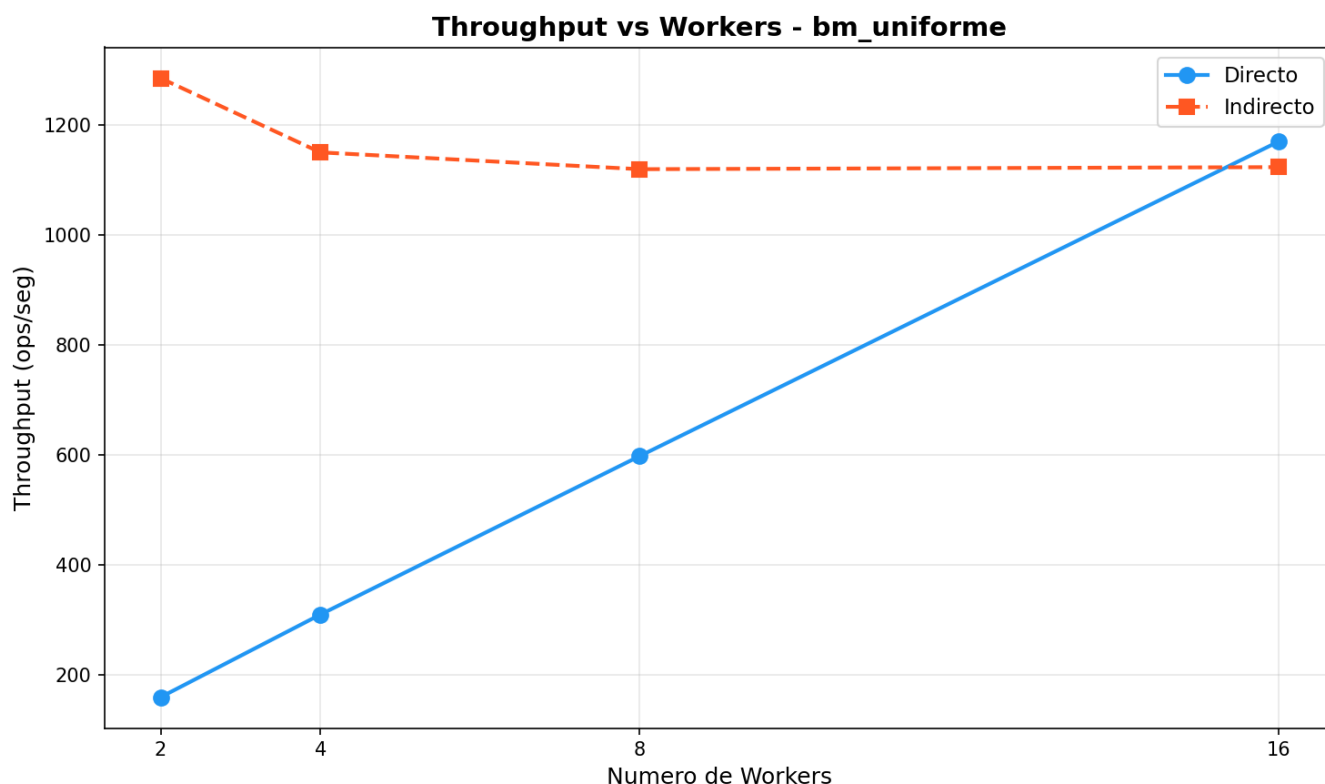


Fig. 6 - Throughput vs Workers (distribución uniforme)

- **Modelo Directo (Pyro4):** Muestra una progresión lineal, escalando limpiamente desde las 160 ops/seg hasta alcanzar su máximo de casi 1.170 ops/seg con 16 workers. El incremento se debe a que la distribución uniforme evita bloqueos centralizados y, dado que 41.000 peticiones son denegadas, Redis resuelve las colisiones instantáneamente mediante SET NX devolviendo falso. Al

procesarse respuestas negativas sumamente ligeras en memoria, las llamadas RPC se liberan muy rápido hacia el cliente, maximizando la eficiencia de los workers.

- **Modelo Indirecto (RabbitMQ):** Alcanza su máximo con 2 *workers* (~1.280 ops/seg). Al pasar a 4 *workers*, sufre una leve degradación porque el coste de coordinación del middleware supera el beneficio de la paralelización. Al pasar a 8 y 16 *workers*, el rendimiento se aplana por completo (~1.120 ops/seg) debido a una saturación estructural: RabbitMQ alcanza su límite máximo de gestión de mensajes y ACKs concurrentes, convirtiéndose en un cuello de botella por muchos hilos que se añadan.

Conclusión: El modelo indirecto es inicialmente más rápido debido a la asincronía de RabbitMQ, pero demuestra que la distribución uniforme estanca su rendimiento a partir de los 4 *workers* por la saturación en la gestión de ACKs del middleware, permitiendo que el modelo directo lo supere con 16 *workers* gracias al descarte instantáneo de peticiones en Redis que libera rápido las llamadas RPC.

Benchmark 4: Hotspot (60.000 peticiones, 80% en 5% de asientos)

60.000 peticiones donde el 80% se concentra en el 5% de los asientos (1.000 asientos). Se consiguen ~10.000 compras exitosas.

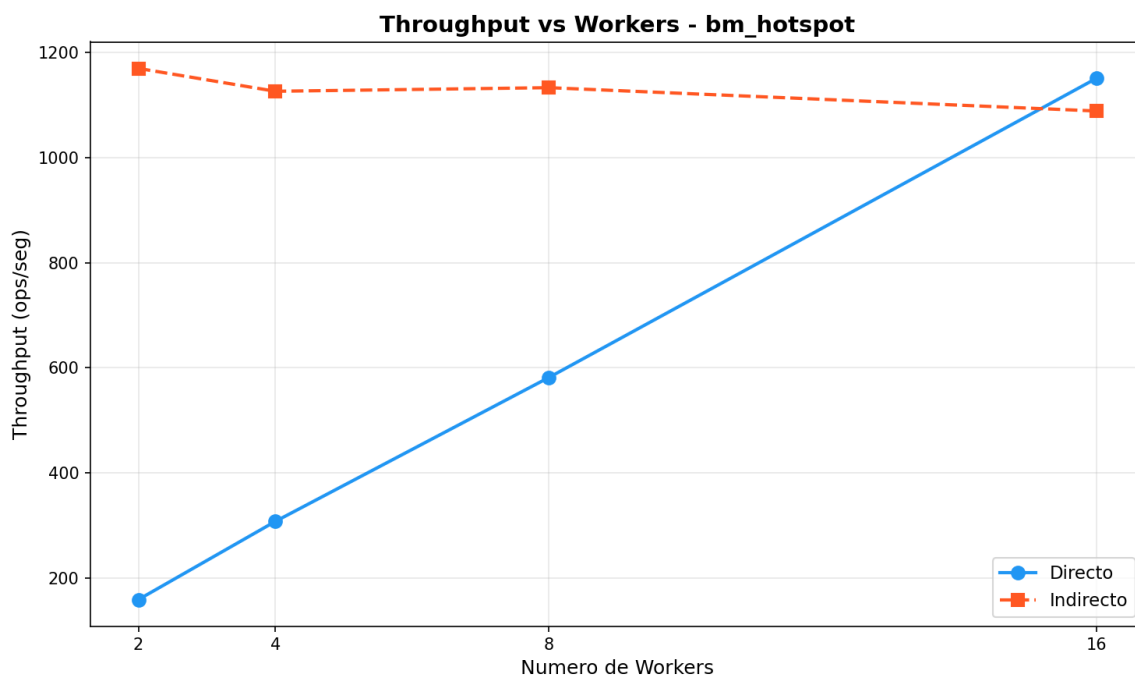


Fig. 7 - Throughput vs Workers (hotspot 80/5)

- **Modelo Directo (Pyro4):** Escala de forma lineal desde las 160 ops/seg hasta casi 1.150 ops/seg con 16 workers. Al ser síncrono, las peticiones llegan filtradas porque cuando chocan contra los hotspots, Redis las rechaza instantáneamente con SET NX sin coste. Esto libera rápido la llamada y permite que el rendimiento suba linealmente al añadir workers.
- **Modelo Indirecto (RabbitMQ):** Alcanza su máximo con 2 *workers* (~1.170 ops/seg. A diferencia del caso uniforme, entre **4 y 8 workers** experimenta un sutil repunte gracias a que disponer de más hilos ayuda a procesar y descartar más rápido el grueso de peticiones rechazadas en los asientos calientes. Finalmente, a los 16 *workers* vuelve a estabilizarse a la baja debido a que la saturación estructural de RabbitMQ.

Conclusión: El modelo indirecto es inicialmente más rápido por su asincronía, pero el hotspot concentrado estanca su rendimiento en torno a las 1.100 ops/seg. A diferencia del caso uniforme, el descarte masivo sobre pocos asientos calientes permite un ligero repunte a los 8 workers, pero la saturación estructural de ACKs en RabbitMQ termina anulando la escalabilidad y permite que el modelo directo lo supere con 16 workers.

Comparativa general de arquitecturas

Throughput vs Workers - Todos los benchmarks

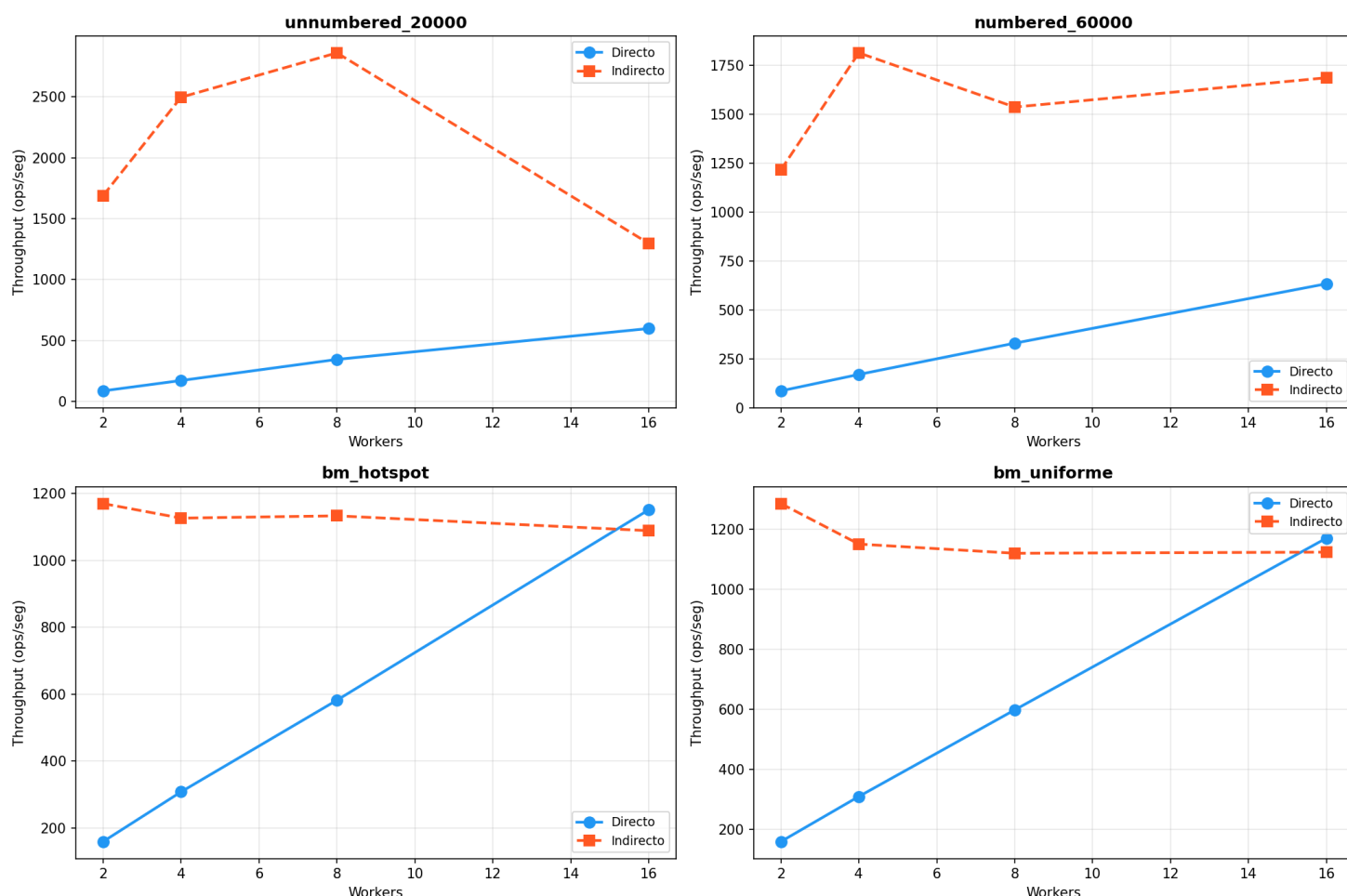


Fig. 8 - Comparativa combinada de throughput para todos los benchmarks

Mientras que el **Modelo Directo (Pyro4)** muestra una escalabilidad lineal y predecible en todos los escenarios porque el sincronismo de la red lo hace inmune al tipo de carga, logrando superar al modelo asíncrono con 16 workers en las pruebas masivas de 60.000 peticiones gracias a que el descarte instantáneo en Redis (SET NX) libera las llamadas RPC a gran velocidad; el **Modelo Indirecto (RabbitMQ)** domina con rotundidad en escenarios de menor volumen u operaciones atómicas simples al eliminar el bloqueo del cliente, pero sufre una saturación estructural ante ráfagas extremas donde el coste de gestión interna de los ACKs concurrentes y el reparto de mensajes fija un techo físico insuperable que aplan su rendimiento a partir de los 4 workers.

4. Análisis

- **Rendimiento:**

La comunicación indirecta (RabbitMQ) supera a la directa en escenarios estándar porque la asincronía elimina el bloqueo del cliente. Sin embargo, ante cargas masivas (60.000 peticiones), la comunicación directa (Pyro4) iguala o supera al modelo indirecto con 16 workers, ya que el descarte inmediato de errores en Redis (SET NX) libera las llamadas sin saturar la red.

- **Escalabilidad:**

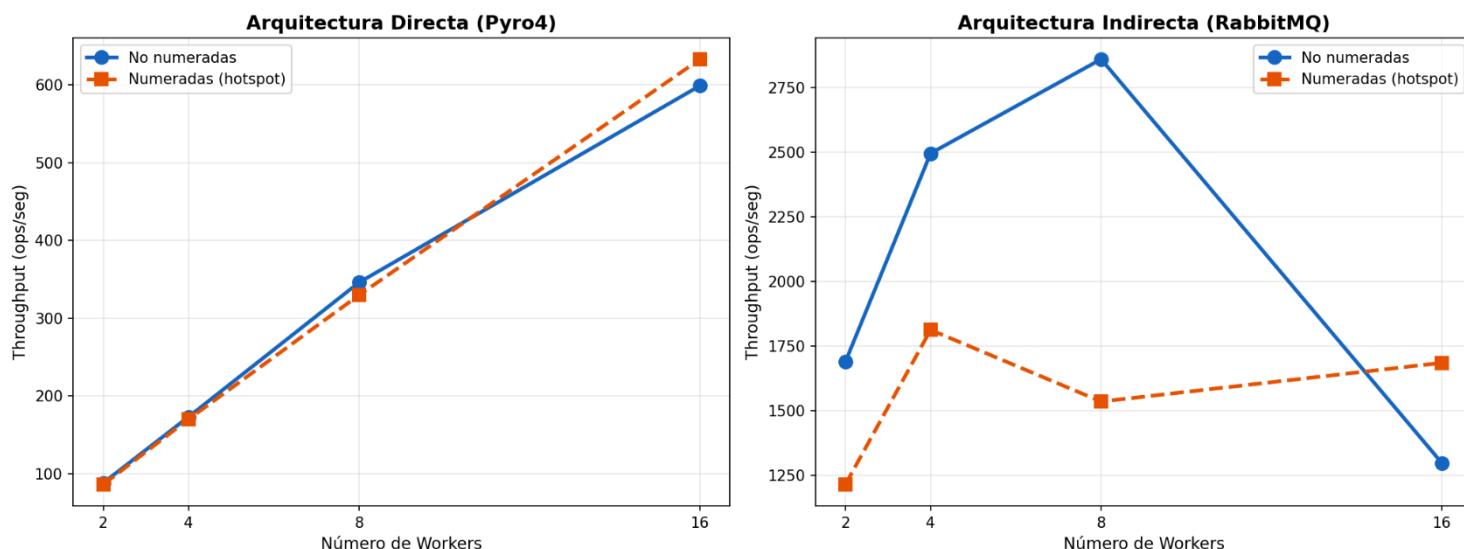
El modelo directo escala de forma lineal hasta los 16 workers en todos los casos. Pero el modelo indirecto presenta una saturación estructural a partir de los 4 workers; añadir más hilos no mejora el rendimiento debido a que RabbitMQ alcanza su límite de gestión de ACKs concurrentes y reparto de mensajes, convirtiéndose el middleware en el cuello de botella.

- **Impacto de la contención (Hotspot):**

La consistencia está garantizada por Redis mediante operaciones atómicas. El flujo síncrono hace al modelo directo inmune a la distribución de la carga. Pero en el modelo indirecto la concentración en un único asiento satura Redis con 8 workers, mientras que una contención repartida (uniforme) acelera el vaciado de la cola gracias al descarte rápido en memoria.

Comparativa numbered vs unnumbered

Comparativa: Entradas No Numeradas vs Numeradas



- **Entradas No Numeradas:** Permiten el rendimiento máximo con 8 workers (~2.850 ops/seg) al realizar solo compras exitosas. Al subir a 16 workers cae el rendimiento porque se satura a Redis con demasiadas escrituras simultáneas y se ahoga la cola de RabbitMQ al enviarle miles de confirmaciones (ACKs) al mismo tiempo.
- **Entradas numeradas:** De 2 a 4 workers el rendimiento sube porque a más workers más rendimiento hasta que a partir de los 4 (de 4 a 8 workers) el funcionamiento del propio sistema satura la comunicación. El número 16 estabiliza de nuevo el rendimiento ya que al tener el doble de workers que 8 torna a compensar la velocidad de la comunicación.

5. Conclusiones:

La comunicación directa con Pyro4 ofrece un modelo sencillo y predecible que escala adecuadamente para cargas moderadas. Sin embargo, su naturaleza síncrona la hace vulnerable a fallos y poco eficiente para picos de demanda.

La comunicación indirecta con RabbitMQ demuestra ser superior en rendimiento y resiliencia, especialmente para entradas no numeradas donde no hay contención por recursos. Para entradas numeradas con hotspots, el cuello de botella es Redis, no el middleware, por lo que añadir más workers no mejora el throughput.

En conclusión, para la venta de entradas de alta demanda en entornos reales, se recomienda la arquitectura indirecta (RabbitMQ) debido a su mayor rendimiento, mejor resiliencia (persistencia de mensajes, ACKs) y capacidad para manejar picos de tráfico. La arquitectura directa (Pyro4) es adecuada para sistemas más simples con carga moderada donde se prioriza la facilidad de implementación sobre el rendimiento bruto.

Tabla 1 - Comparativa de arquitecturas: Directa vs Indirecta

Aspecto	Comunicación Directa (Pyro4)	Comunicación Indirecta (RabbitMQ)
Rendimiento no numeradas	Bajo (~600 ops/s con 16w)	Alto (~2859 ops/s con 8w)
Rendimiento numeradas	Bajo (~633 ops/s con 16w)	Medio (~1800 ops/s)
Escalabilidad	Buena (~7x speedup con 16w)	Buena solo en no numeradas, limitada en numeradas
Resiliencia	Baja (fallo de worker = pérdida)	Alta (ACKs, persistencia, idempotencia)
Complejidad	Baja (RPC directo)	Media (broker, colas, ACKs)
Mejor para	Sistemas simples con baja carga	Sistemas de alta concurrencia y misión crítica